

CHL5233: Quarto in-class assignment

Alex Chen

For this in-class assignment, you will work in a group to create your **individual** Quarto document. Please consult your classmates and [Tutorial: Authoring](#) webpage by [Quarto](#). You can also use the Internet including ChatGPT. The TA and I are also available for any questions.

When you are finished, render the document to PDF. Please carefully **proof read** and make edits as needed. **Commit** and **push** the edits to this file and the new PDF file to your **new repository**.

Create a new repository

- Under your GitHub account, create a **new** repository called `armed_conflict`
 - Throughout the course, we will use this repository to **reproduce** and **replicate** the analysis in the paper
- Create **two** folders
 - `data`
 - `reports`
- Save `countries.txt` and `regions.txt` into `data/raw/` subfolder
- Save `quarto_inclass.qmd` into `reports/` folder
- Eventually, your repository will have a very organized structure like this:

Example of data science organization

```
armed_conflict/  
  README.md  
  data/  
    raw/  
    processed/  
  scripts/  
    01_import.R  
    02_clean.R
```

```
03_analysis.R
outputs/
  figures/
  tables/
reports/
  analysis.qmd
```

Edit this file to create a PDF document using Quarto

The R code chunk below includes the R packages required to knit this document. This is often called a **setup** code. We want the code to run, but we don't want the final document to show the code or results. Edit the code chunk so that the code will run but the code and results (including all warnings and messages) are hidden from the final PDF output.

For this exercise, we will use the following two data sets.

1. **countries.txt**: A data set that contains a list of countries used in the paper “Implications of armed conflict for maternal and child health: A regression analysis of data from 181 countries for 2000–2019” and their three-digit ISO (International Organization for Standardization) code.
2. **regions.txt**: A data set that contains a list of all major countries in the world and their
 - a. ISO code (**country_code**)
 - b. region (**region**)
 - c. sub-region (**sub_region**)
 - d. numerical region code (**region_code**)
 - e. numerical sub-region code (**sub_region_code**)

The **final goal** is to display a **bar chart** that depicts how many countries are in each region.

Read in data and create an analytical data set

First, edit the code chunk below that reads in **countries.txt** to also display the number of rows and columns of the data.

```
countries <- read.table("../data/raw/countries.txt", header = TRUE)
nrows_coun <- nrow(countries)
ncols_coun <- ncol(countries)
sprintf("Number of rows: %d", nrows_coun)
```

```
[1] "Number of rows: 186"
```

```
sprintf("Number of columns: %d", ncols_coun)
```

```
[1] "Number of columns: 2"
```

```
sprintf("Number of unique countries: %d", length(unique(countries$country_name)))
```

```
[1] "Number of unique countries: 186"
```

Answer: From the code output above, we see that the `countries` dataset contains 186 unique countries (same as the number of rows).

Second, read in `regions.txt` and also write a sentence that states how many countries are included in the data set.

```
regions <- read.table("../data/raw/regions.txt", header = TRUE)
nrows_reg <- nrow(regions)

sprintf(
  "# of unique countries in regions.txt: %d",
  length(unique(regions$country_name))
)
```

```
[1] "# of unique countries in regions.txt: 249"
```

Answer: We see that the `regions` dataset contains 249 unique countries; more than that of `countries.txt`.

Third, subset the `regions` data set to only include countries from the `countries` data set.

```
regions_filt <- regions[regions$country_name %in% countries$country_name, ]

sprintf("Number of rows in 'regions_filt': %d", nrow(regions_filt))
```

```
[1] "Number of rows in 'regions_filt': 164"
```

```
sprintf("Number of rows in original 'countries' dataset: %d", nrow(countries))
```

```
[1] "Number of rows in original 'countries' dataset: 186"
```

```
sprintf(
  "Does regions_filt have all countries from 'countries' dataset: %s",
  nrow(regions_filt) == nrow(countries)
)
```

```
[1] "Does regions_filt have all countries from 'countries' dataset: FALSE"
```

```
sprintf(
  "Is 'regions_filt' subset of 'countries': %s",
  all(regions_filt$country_name %in% countries$country_name)
)
```

```
[1] "Is 'regions_filt' subset of 'countries': TRUE"
```

Tabulate countries by region and sub-region

Next, we want to find out how many countries are in a region. Use the `group_by()` and `summarise()` functions in the `dplyr` package to count the number of countries in each region. Then use the `kable()` function in the `knitr` package to create a simple table with the column headers, “Region” and “Count”.

```
# Filter by region, then create column for count of countries per region
ncountries_reg <- regions_filt %>%
  group_by(region) %>%
  summarise(num_countries = n())
)

kable(
  head(ncountries_reg),
  col.names = c("Region", "Count"),
  align = c("l", "l"),
  caption = "Number of Countries per Region"
)
```

Table 1: Number of Countries per Region

Region	Count
Africa	48
Americas	31
Asia	39
Europe	36

Region	Count
Oceania	10

Create a similar table that shows the number of countries in each **sub-region**.

```
# Filter by subregion, then create column for count of countries per subregion
ncountries_sr <- regions_filt %>%
  group_by(sub_region) %>%
  summarise(num_countries = n())

kable(
  head(ncountries_sr),
  col.names = c("Sub-region", "Count"),
  align = c("l", "l"),
  caption = "Number of Countries per Sub-region"
)
```

Table 2: Number of Countries per Sub-region

Sub-region	Count
Australia and New Zealand	2
Central Asia	5
Eastern Asia	3
Eastern Europe	8
Latin America and the Caribbean	30
Melanesia	4

Data visualization using ggplot

Here, we will use `geom_bar()` function within `ggplot()` to create a bar chart that displays the **number of countries** in each **sub-region**.

1. Color code each sub-region by **region**.
2. Create bars that run horizontally where sub-region names appear along the y-axis.
3. Order the bars such that the sub-region with the most number countries appear at the top and the sub-region with the least number of countries appear at the bottom.

```
library(forcats)

ncountries_sr_plot <- ggplot(
  data = regions_filt,
```

```

mapping = aes(x = fct_rev(fct_infreq(sub_region)), fill = region)
) +
geom_bar() +
labs(
  x = "Sub-region",
  y = "Number of Countries",
  title = "Number of Countries per Sub-region",
  fill = "Region"
) +
coord_flip()

ncountries_sr_plot

```

